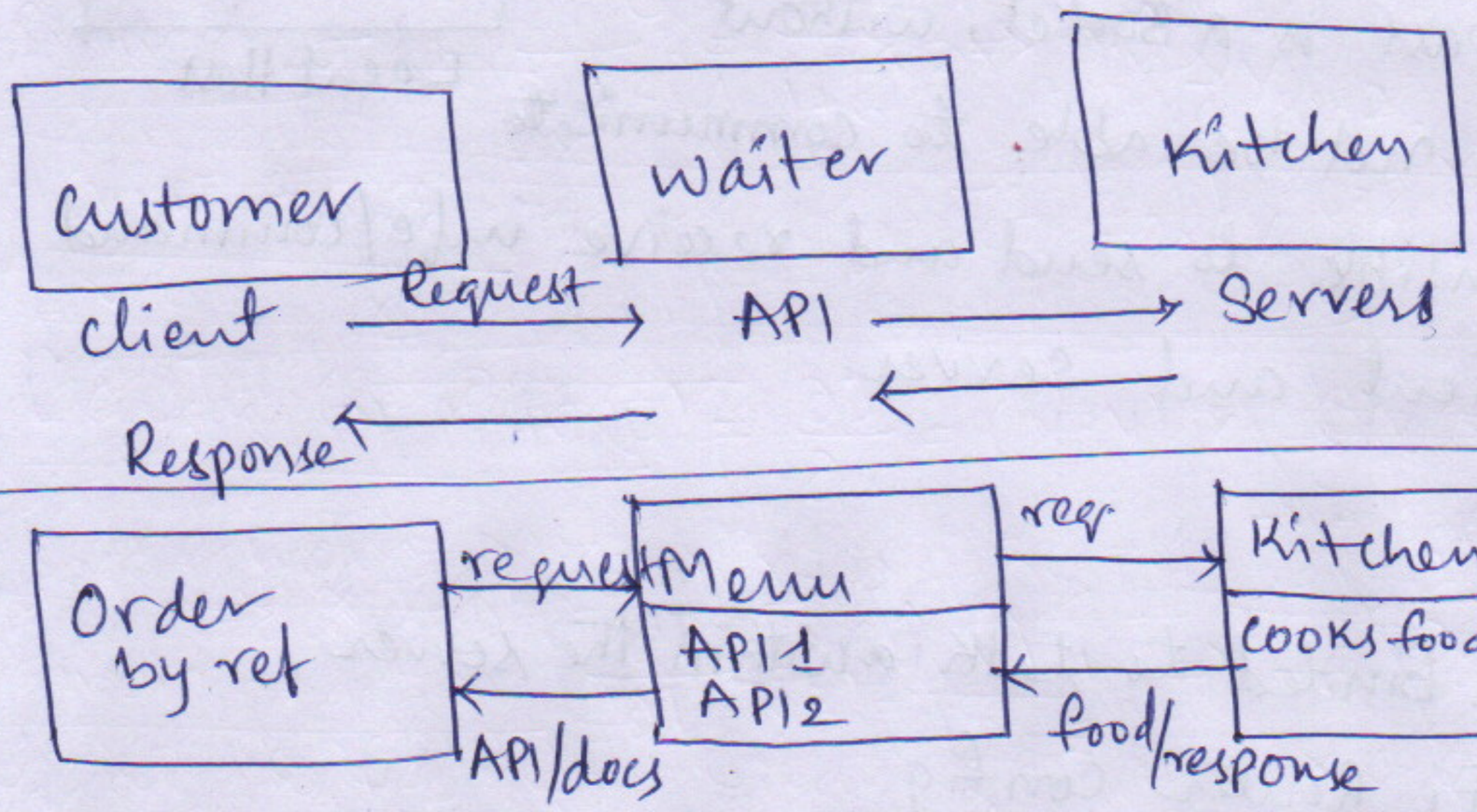
FAST API Notes

→ What is a web Server and client

→ what's API, who creates and consumes it

→ Analogy of serving an API, with a restaurant example




```
while True:
    obj = request()
    process obj
```

→ API is agreed upon contract, ~~binds~~

Servers: with IP address

→ Web Server (http) →
→ Database (postgresql)
→ SSH (remote terminal)
→ FastAPI

} different ports

→ different ports used are:
80, 5432, 22

Range: 0 to 65,535

< 1024 port is used for priviled

→ netstat -ano → show active connections with ports.

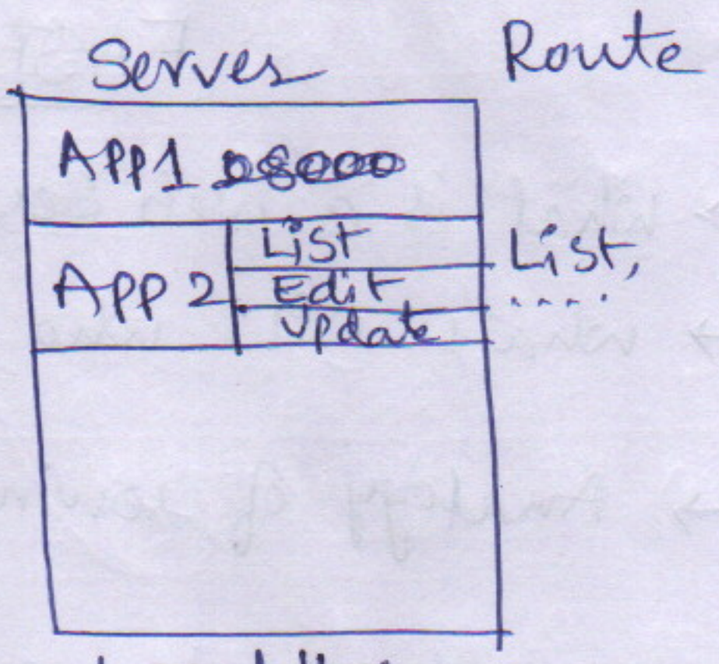
→ Think, IP address is like street name
port is " door number

→ within the server, it is called "localhost/127.0.0.1"

http://localhost:8000

App1 access: localhost:8000/list

Server address → Post → Route



→ **Socket :** Call past is a socket, without which we will not be able to communicate
→ it is responsible to send and receive info/command among client and server

→ **Bind :**
→ local host is binded to talk within the server

Example: WiFi Router config

Binding posts:
- 127.0.0.1 → itself (secure)
- 192.168.0.1 → WiFi n/w interface (restricted)
- 200.128.0.5 → Internet / public IP (Open access)
- 0.0.0.0 → Open its post to every active i/f on the machine

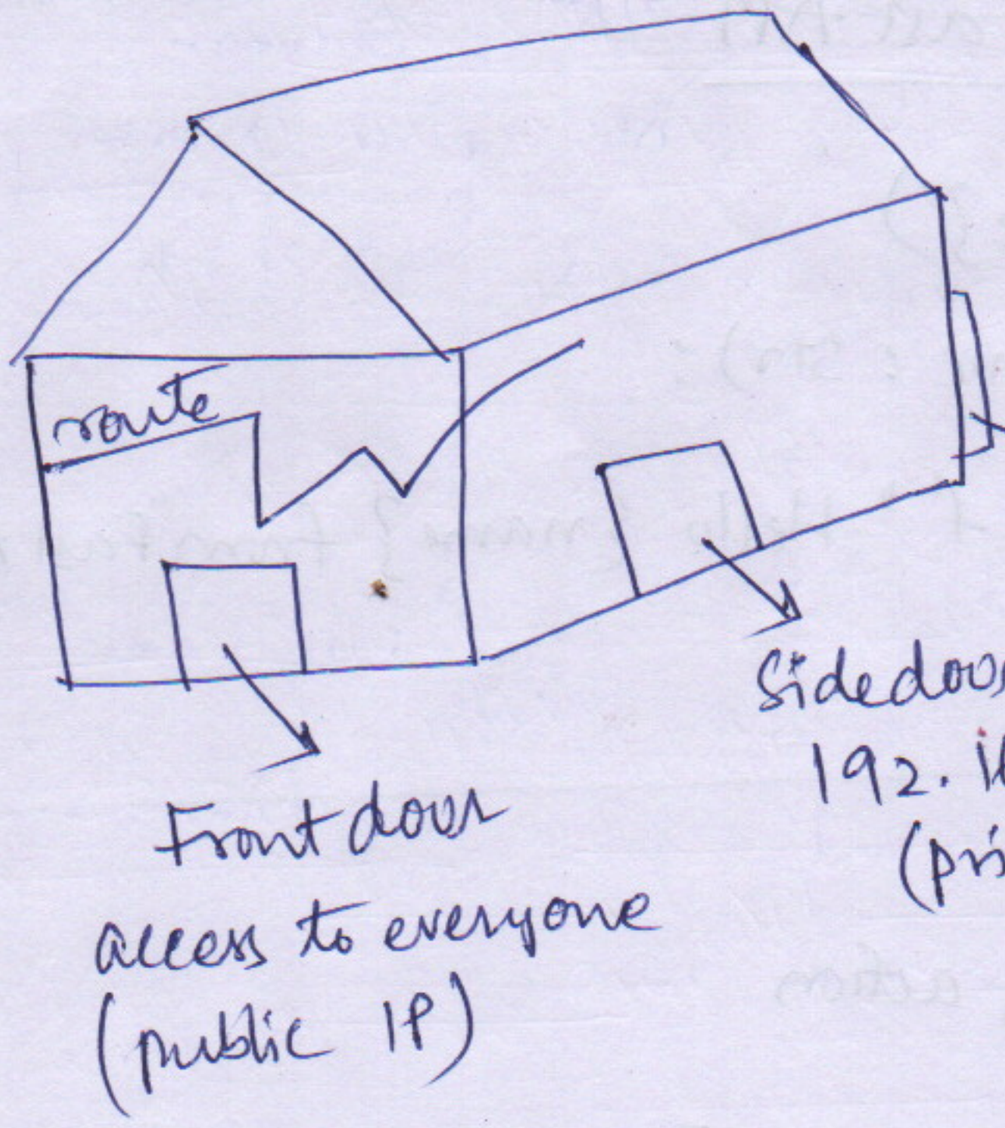
→ For FastAPI application building, we need two libraries

like doorman
- UVICORN → Server program → Socket creation, binding, listening and accepting
- FastAPI → Framework →
  - → What happens when a request arrived
  - → Holds the menu

Notice when someone walks to restaurant. No need to know about menu but send to waiter in Restaurant analogy.

Server, client, request, response, API, ip-address, port, socket, route

Tools: postman, apidog for testing the app



→ To accept ~~connec~~ request from local, private and public access bind to 0.0.0.0 ip

---

Basic python program

fastapi.py:

```
# run uvicorn fastapi:app --port 8000 --reload
~~import~~ from ~~fastapi~~ import FastAPI

app = FastAPI()  # app can be anyname

# route / endpoint / path operation
@app.get("/")
def read_root():
    return {"message": "Hello from FastAPI"}  # always return in dictionary format

if __name__ = "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

→ Different route in FastAPI

```
@app.get("/say_hello/{name}")
def say_hello_python(name: str):
    return {"message": f"Hello {name} from FastAPI"}
```

HHtp request :

1) method → what kind of action
   - GET → to get data
   - POST → get data ~~along~~ by sharing extra meta data
   - PUT → replace data
   - DELETE → remove data

2) Path → part of url, what you're asking

   Example: "/", "/say-hello", "/predict", "/predict/{val}"

3) Path Parameter: "/predict{val}"

   Path param

   ↳ pieces of path with variables ("/items/2")

4) Query parameters: extra key:value pairs appended to url after "?".

   Example: '/search?q=shoes&limit=5'

5) Headers: meta data about the request, send separately from main content

   Example: 'content-Type': application/json'

6) Body: The actual data being sent with POST / PUT

   Example: DATA(PUT: {sepal_length: 3.4})

STATUS CODES

→ 3 digit codes summarizing the request.

- 2XX → Success (200 OK, 201 created)
- 3XX →
- 4XX → Client side error.
  - → 400 Bad request
    - 404 Not Found
    - 422 Unprocessable Entity
- 5XX → The Server side Error
  - 500 internal Server Error

} Http response

Headers: Server response to client request

Content_Type : application/json

Body: The actual data that was requested by client

---

fastapi2.py

```
from fastapi import FastAPI

app = FastAPI()

FAKE_ITEMS_DB = {
    1: {"name":"Keyboard", "price": 49},
    2: {"name": "monitor", "price": 400}
}
@app.get("items/{item_id}")
def read_root(item_id:int):
    return FAKE_ITEMS_DB[item_id]
```

CURL commands

→ curl http://localhost:8000/all items

Syntax:

curl -X GET/POST "PATH?key=value"
-H "Content-Type: application/json" -d "data/body

docs

http://localhost:8000/docs

Query Parameters

```
@app.get("/search")
def search_items(q: str|None, limit: int!0
    if q is None or limit is None:
        raise HttpException(status_code=404, detail="
                                Need to pass Q param")
    else:
        matches = [item for item in FAKE_ITEMS_DB.values()
                   if q.lower in item["name"]]
        return {"query": q, "limit": limit, "results":
                matches[:limit]}
```

Post Command

```
@app.post("/items", status_code = 201)
def create_item(item: dict):
    new_id = max(FAKE_ITEMS_DB.keys()) +1
    FAKE_ITEMS_DB[new_id] = item
    return {"id": new_id, **item}
```

curl -X POST "localhost:8000/items" -d "{\"name\":\"mic\"}"

Pydantic

pydantic.py

```
from pydantic import BaseModel
from fastapi import FastAPI

app = FastAPI(title="FastAPI")

FAKE_ITEMS_DB = {
    1: {"name": "keyboard", "price": 40}
    2: {"name": "mic", "price": 50}
}
```

```
class Item(BaseModel):
    name : str
    price : float
```

```
class Prediction(BaseModel):
    prediction : str
    confidence : float
```

```
class
IrisFeatu
(BaseModel
setosa:str
```

```
@app.post("/items", status_code=201, response_model=
                                          Prediction)
~~def create_item(item: Item):~~
def predict_iris(features: IrisFeatures):
    response = iris_model(features.sepal_len ...)
    return ~~response~~ Prediction(response[0], response[1])
```

→ Why the workshop → multi skilling, aim to deliver RTPO AI ML needs

→ what is for you → RAIL, Hands on Lab, support

→ what will be covered

→ Foundational AI ML + Labs, concepts on AI ML, deploy mode

→ what is out of scope

Python, ~~stat~~ Mathematics → Stats, calculus, trigonometrics

~~Advanced~~ ML, Image processing, NLP, Gen AI

Visualisation, EDA, FE

→ Capstone project and submission, certificate

→ Topic Team

PoV

→ How many of you from cs background

→ How many of you know or built models before

→ How many of you have coded in python.

→ Where do you think ML will be used

